

Audit Logging System

This document describes the audit logging system for HIPAA compliance, covering security events, data access logging, and document audit trails.

Overview

The platform implements audit logging designed to align with HIPAA Security Rule §164.312(b) requirements. This is a **prototype implementation** that aims to follow HIPAA best practices but has not been formally audited or certified. All access to Protected Health Information (PHI) and security-relevant events are logged to the database with full context.

Audit Tables

1. audit_logs - PHI Data Access

Tracks all access to Protected Health Information.

Column	Type	Description
id	Integer	Primary key
table_name	String(100)	Database table accessed (e.g., <code>clients</code> , <code>client_documents</code>)
record_id	Integer	Primary key of the accessed record
action	String(50)	<code>create</code> , <code>read</code> , <code>update</code> , or <code>delete</code>
old_values	JSONB	Previous values (for updates/deletes)
new_values	JSONB	New values (for creates/updates)
staff_id	Integer	User who performed the action
ip_address	String	Client IP address
user_agent	Text	Browser/app user agent
created_at	DateTime	Timestamp of the action

2. security_events - Security Event Log

Tracks authentication and security-related events.

Column	Type	Description
id	Integer	Primary key
event_type	String(100)	Type of security event (see below)
severity	String(20)	<code>info</code> , <code>warning</code> , <code>error</code> , or <code>critical</code>
description	Text	Human-readable description

Column	Type	Description
staff_id	Integer	User involved (if applicable)
ip_address	String	Client IP address
user_agent	Text	Browser/app user agent
additional_data	JSONB	Additional context data
created_at	DateTime	Timestamp of the event

Event Types

Event Type	Severity	Description
login_success	info	Successful user login
login_failure	warning	Failed login attempt
logout	info	User-initiated logout
session_timeout	info	Client-side inactivity logout
session_expired	info	Session expired
password_change	info	User changed their password
admin_password_reset	warning	Admin reset a user's password
unauthorized_access	error	Attempt to access unauthorized resource

3. client_document_audit_log - Document Audit Trail

Tracks all actions on client documents for compliance.

Column	Type	Description
id	Integer	Primary key
document_id	Integer	Original document ID
resulting_document_id	Integer	New document version (if edit created new version)
performed_by	Integer	Staff who performed the action
performed_at	DateTime	Timestamp of the action
changes_made	Boolean	Whether content was modified
action_type	String(50)	creation, edit, review, new_template, archived
notes	Text	Additional notes or comments

4. shift_task_status_history - Task Status Changes

Tracks all changes to shift task statuses for care documentation integrity.

Column	Type	Description
<code>id</code>	Integer	Primary key
<code>shift_task_status_id</code>	Integer	The task status record
<code>old_status_id</code>	Integer	Previous status
<code>new_status_id</code>	Integer	New status
<code>changed_by</code>	Integer	Staff who made the change
<code>changed_at</code>	DateTime	Timestamp of the change
<code>notes</code>	Text	Change notes

AuditService Usage

The `AuditService` class in `server/services/audit.py` provides methods for logging audit events.

Initialization

```
from services.audit import AuditService

audit = AuditService(
    db=db,
    user_id=current_user.id,
    ip=request.state.client_ip,
    user_agent=request.headers.get("user-agent")
)
```

Logging Data Access

```
# Log a read operation
audit.log_data_access("read", "clients", client_id)

# Log an update with old/new values
audit.log_data_access(
    action="update",
    table_name="clients",
    record_id=client_id,
    old_values={"phone": "555-1234"},
    new_values={"phone": "555-5678"}
)

db.commit() # Caller must commit
```

Logging Security Events

```
# Log a security event
audit.log_security_event(
    event_type="login_success",
    severity="info",
    description=f"User {staff.id} logged in successfully",
    additional_data={"method": "password"}
)

db.commit() # Caller must commit
```

Compliance Notes

Note: This is a prototype implementation. While we aim to follow HIPAA best practices, this system has not been formally audited or certified for HIPAA compliance.

- 1. **Retention:** Audit logs are retained via the backup system. Encrypted daily backups are kept for **6 years** (backups older than 2190 days are automatically deleted). The audit tables in the live database grow indefinitely and are not pruned.
- 2. **Immutability:** Audit log entries are **append-only**. No UPDATE or DELETE operations are permitted on audit tables.
- 3. **Automatic Logging:** Security events are automatically logged by the authentication system for:
 - Login attempts (success and failure)
 - Logout events
 - Password changes
 - Session timeouts
- 4. **Document Versioning:** Client documents use a "never-overwrite" pattern. Edits create new versions, preserving the complete audit trail.
- 5. **Backup Integration:** Audit logs are included in the encrypted daily backups (see [BACKUP_RECOVERY.md](#)).

Internal System Tables

These tables are managed by backend services and have no direct API endpoints:

Table	Purpose
audit_logs	PHI access logging
security_events	Security event logging
user_sessions	Active session tracking

Table	Purpose
encryption_keys	Key management
staff_permissions	Granular permissions